

8 Week Plan for Hari Priya

Phase 1: Core Computer Science Foundations

Week 0: Environment Configuration & Git Mechanics

- **Focus Area:** Professional developer setup, shell navigation, and strict local source control habits.
- **Expected Learning Outcomes:**
 - Configure a standard development environment using VS Code.
 - Execute explicit version control states using the command line (git add, git commit, git status).
 - Navigate local directories exclusively through terminal commands (cd, ls, mkdir, pwd).
- **Daily Execution Plan:**
 - **Day 1:** Install VS Code. Set up a clean, accessible workspace directory. Learn fundamental Bash/zsh commands.
 - **Day 2:** Initialize a global Git profile. Run through modules 1–10 of gitimmersion.com.
 - **Day 3:** Create a public GitHub account. Practice creating a local repository, linking a remote origin, and pushing code changes.
 - **Day 4:** Solve 1–2 LeetCode Easy problems locally in Python. Commit the exact initial logic state.
 - **Day 5:** Refactor the LeetCode solutions to improve readability. Push the final cleaned script states to the remote GitHub repository.
- **Weekly Deliverable:** A public GitHub repository featuring 3–5 LeetCode Easy solutions, demonstrating a multi-commit history with professional, concise commit messages.

Week 1: Core Data Structures (Python Arrays & Strings)

- **Focus Area:** Mastering Python's native collections (lists, dictionaries) and managing basic disk File I/O operations.
- **Expected Learning Outcomes:**
 - Manipulate multi-line strings, extract target segments, and clear empty spacing natively.
 - Map, traverse, and query key-value data structures (Python dict).
 - Programmatically open, read, and stream local flat files (.txt, .md) without using external libraries.
- **Daily Execution Plan:**

- **Day 1:** Study Python list indexing, slicing operations, and native array methods (`.append()`, `.pop()`, `.extend()`).
- **Day 2:** Study Python dictionary structures: key insertion, safe lookups via `.get()`, and looping through `.items()`.
- **Day 3:** Learn built-in file operations using the context manager syntax (with `open()` as file:). Practice reading small text structures.
- **Day 4:** Solve 2 LeetCode Easy array/string manipulation problems (e.g., Two Sum or Valid Palindrome).
- **Day 5:** Write a script that reads a sample 10-line text file, strips trailing whitespace from each line, and prints the output to the console.
- **Weekly Deliverable:** 3–4 new LeetCode Easy scripts pushed to the repository, alongside a standalone script that successfully targets, reads, and prints a local multi-line text file.

Week 2: Algorithmic Complexity & Data Streaming

- **Focus Area:** Evaluating algorithmic time/space constraints and implementing memory-efficient lazy evaluation patterns.
- **Expected Learning Outcomes:**
 - Evaluate Python code blocks using formal Big O notation rules ($O(1)$, $O(n)$, $O(n^2)$).
 - Contrast loading whole datasets into memory vs. processing them sequentially using Generators (`yield`).
 - Track active system memory usage patterns when handling string streams.
- **Daily Execution Plan:**
 - **Day 1:** Study the mathematical foundations of Big O. Practice calculating loop iterations and scaling bounds.
 - **Day 2:** Analyze the memory difference between `file.read().splitlines()` ($O(n)$ space complexity) and looping over a file handle directly ($O(1)$ space complexity).
 - **Day 3:** Learn Python Generator functions. Write a basic functional wrapper that uses the `yield` keyword to return single values lazily.
 - **Day 4:** Refactor the file-reading script from Week 1 into a pure stream generator that processes lines sequentially on demand.
 - **Day 5:** Review previous LeetCode code blocks. Write a formal time and space complexity audit directly into the file headers.
- **Weekly Deliverable:** A refactored file-reading script utilizing a custom Python generator (`yield`), with explicit time/space complexity calculations documented in the comments of all repo scripts.

Week 3: Structured Data Formats & Serialization (JSON)

- **Focus Area:** Persisting application states to the local disk without a relational database engine.
- **Expected Learning Outcomes:**
 - Distinguish raw text structures from formal serialized string formats.
 - Serialize native Python dictionaries into JSON strings (`json.dumps`, `json.dump`).
 - Deserialize flat storage files back into mutable Python memory objects (`json.loads`, `json.load`).
- **Daily Execution Plan:**
 - **Day 1:** Study the structural rules of the JSON format (keys, nested arrays, specific datatypes).
 - **Day 2:** Learn Python's native json library. Practice converting a nested Python dictionary into an inline string configuration.
 - **Day 3:** Build a write-pipeline: take an in-memory dictionary array and save it to a persistent local file named `tasks.json`.
 - **Day 4:** Build a read-pipeline: open `tasks.json`, parse its contents back into active memory, modify a key value, and save it back to disk.
 - **Day 5:** Combine these actions into a modular script containing `clear_load_data()` and `save_data()` functions.
- **Weekly Deliverable:** A functional state-tracking script that reads a local `tasks.json` file on execution, modifies a task array in memory, and writes the updated array back to disk.

Phase 2: AI Integration & CLI Engine Synthesis

Week 4: Environment Isolation & Security

- **Focus Area:** Isolating dependencies via localized execution runtimes and securing sensitive API access variables.
- **Expected Learning Outcomes:**
 - Initialize, activate, and manage isolated local execution runtimes via Python Virtual Environments (`venv`).
 - Enforce dependency state management using clean output locks (`requirements.txt`).
 - Isolate runtime configuration parameters and secret tokens using external environment files (`.env`).
- **Daily Execution Plan:**
 - **Day 1:** Study the risks of global Python package pollution. Initialize a local virtual runtime environment (`python -m venv .venv`).

- **Day 2:** Practice environment activation/deactivation mechanics across various terminal contexts.
- **Day 3:** Create a standard .gitignore file to ensure the .venv/ runtime directory is permanently blocked from source control tracking.
- **Day 4:** Install the python-dotenv package. Create a local .env file holding a placeholder token value (MOCK_API_KEY=12345).
- **Day 5:** Write an isolated script that safely targets, extracts, and prints the masked environment token via os.environ.
- **Weekly Deliverable:** A verified, active virtual runtime environment with a locked requirements.txt file and a script that successfully prints an encrypted token from a git-ignored .env file.

Week 5: AI SDK Fundamentals

- **Focus Area:** Consuming third-party AI service APIs and orchestrating foundational network-bound text completion requests.
- **Expected Learning Outcomes:**
 - Install and configure foundational developer packages (LangChain core packages or standard OpenAI/Hugging Face SDKs).
 - Construct basic text prompt inputs and dispatch synchronous I/O requests over the network.
 - Extract and display raw completion texts from nested API response payloads.
- **Daily Execution Plan:**
 - **Day 1:** Register for an AI developer platform token and add the active credential string directly to the protected .env file.
 - **Day 2:** Install the core LangChain or target provider SDK inside the activated virtual environment.
 - **Day 3:** Write an initialization script that instantiates the language model client, pulling credentials securely from the environment.
 - **Day 4:** Pass a basic text string to the model client and inspect the structural properties of the returned payload object.
 - **Day 5:** Write a simple terminal interface script using Python's built-in input() function to pass live prompts to the model and output the answers.
- **Weekly Deliverable:** A working CLI script that captures user prompt strings directly from the terminal, dispatches them to a live LLM via the SDK, and outputs the plain-text reply.

Week 6: Structured AI Outputs (JSON Schema)

- **Focus Area:** Forcing non-deterministic AI text engines to return rigid, programmatic data schemas.
- **Expected Learning Outcomes:**

- Enforce target schema layouts using prompt engineering instructions or LangChain structured parsers.
- Utilize data validation classes (like Pydantic or basic dictionary contract rules) to verify returned objects.
- Extract a structured string payload from the AI and convert it cleanly into a usable Python dictionary.
- **Daily Execution Plan:**
 - **Day 1:** Study the unpredictable nature of unstructured text generations. Craft a system prompt that explicitly restricts output format rules.
 - **Day 2:** Learn how to use LangChain's structured output mechanisms (or explicit `json_object` parameters) to enforce a target structure.
 - **Day 3:** Design the target code contract schema representing a course breakdown: an array of chronological milestones and specific deliverables.
 - **Day 4:** Write a script that feeds a short sample text syllabus block to the model along with the new formatting instructions.
 - **Day 5:** Verify that the model's string response can be run cleanly through `json.loads()` without raising formatting exceptions.
- **Weekly Deliverable:** An advanced generation script that accepts a multi-line syllabus text block, processes it via the AI SDK, and successfully prints out a verified, structured Python dictionary of milestones and tasks.

Week 7: CLI Engine & State Integration

- **Focus Area:** Connecting independent modules into a unified data processing and storage execution pipeline.
- **Expected Learning Outcomes:**
 - Coordinate multi-file pipelines: read a local file, pass it to an AI parser, and serialize the resulting data objects to disk.
 - Handle edge cases where local file paths are missing or invalid.
 - Maintain clean, decoupled code files by isolating functions into separate logical modules.
- **Daily Execution Plan:**
 - **Day 1:** Move the Week 2 file-streaming functions, the Week 3 JSON storage scripts, and the Week 6 AI processing logic into individual, distinct files.
 - **Day 2:** Create an engine script ([app.py](#)) that imports these individual modules.
 - **Day 3:** Code the primary execution chain: read an external `syllabus.txt` file, pass the data block to the AI module, and catch any structural parsing faults.
 - **Day 4:** Direct the verified dictionary response into the JSON saving pipeline to create a local `course_data.json` storage file.

- **Day 5:** Run manual test validations using different text content styles to confirm that `course_data.json` populates reliably every time.
- **Weekly Deliverable:** A fully integrated pipeline engine script that orchestrates reading a local syllabus file, parsing it via the AI layer, and successfully saving the structured objects into a local `course_data.json` repository file.

Week 8: CLI Polish & User Interaction

- **Focus Area:** Wrapping the core data pipeline in an interactive terminal menu loop to allow state modification and progress tracking.
- **Expected Learning Outcomes:**
 - Build an infinite loop console menu (`while True:`) that processes user input options cleanly.
 - Present nested JSON data collections cleanly using clear formatting patterns.
 - Implement simple item mutation mechanics (finding a task by its unique ID and flipping its status boolean).
- **Daily Execution Plan:**
 - **Day 1:** Code a repeating menu loop that displays clear terminal choice blocks (e.g., 1. View Tasks, 2. Toggle Task Status, 3. Exit).
 - **Day 2:** Implement the display function: read `course_data.json`, loop through the stored arrays, and print milestones alongside a status icon (like `[]` or `[X]`).
 - **Day 3:** Implement the mutation logic: prompt the user for a specific target task ID, locate that object in the list, and invert its completion boolean.
 - **Day 4:** Wire the update logic to rewrite the modified list state back to `course_data.json` instantly upon changes.
 - **Day 5:** Clean up the final codebase layout. Write a conversational [README.md](#) file detailing step-by-step instructions on how to install and run the CLI application.
- **Weekly Deliverable:** The final completed Smart Syllabus CLI Application. The tool reads local files, structures them using AI, saves them to a local JSON file, and provides an interactive terminal menu allowing a user to view milestones and track progress over time.